



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

2026–27, Term-I

Project Problem Statement Submission Form

Section / Team Details

Section No.: 9

Team No.: 22

Project Title: Linux-Service-and-Process-Supervisor

Team Members

Roll Number	Student Name	Signature
2520030252	G. Jothi Ganesh	
2520030179	V. Nitish Chowdhary	
2520030330	P.Santhosh	

1. Abstract

Linux Service and Process Supervisor is a Linux-based Operating Systems and Systems Programming project designed to monitor, manage, and control system processes and services. In a multitasking operating system, multiple processes and background services may need to be started, stopped, monitored, restarted, or terminated efficiently. Managing these processes manually can be difficult and may result in service failures, unresponsive processes, or unnecessary resource consumption.

This project addresses the problem by developing a lightweight process and service supervision system that provides a centralized mechanism for monitoring and controlling processes running in a Linux environment. The project focuses on key Operating Systems and system programming concepts such as **process creation, process identification, process monitoring, process control, process synchronization, signals, inter-process communication (IPC), and service management.**

The proposed solution consists of a Linux-based application developed in C/C++ that maintains information about managed processes and provides operations such as starting, stopping, restarting, monitoring, and terminating processes. Linux system calls and APIs such as `fork()`, `exec()`, `kill()`, `wait()`/`waitpid()`, `getpid()`, and signal-handling mechanisms can be used to implement process supervision and control.

The expected outcome is a functional Linux service and process supervisor capable of monitoring process states, detecting process termination or failure, and performing appropriate control operations. The project provides practical understanding of Linux process management, system calls, signals, process synchronization, and system-level programming while demonstrating how automated supervision can improve process reliability and management.

2. Problem Statement

In a multitasking Operating System, multiple applications and background services may execute simultaneously and require continuous monitoring and management. Processes may terminate unexpectedly, become unresponsive, consume excessive resources, or fail to restart after termination. Without an appropriate supervision mechanism, maintaining the availability and proper execution of important services can become difficult.

This project addresses the problem of **monitoring and controlling Linux processes and services through a centralized supervision mechanism**. The system will provide functionality to identify running processes, monitor their execution status, start and stop processes, restart failed processes, and safely terminate processes when required.

The proposed system uses Linux system calls and process-management mechanisms to communicate with and control processes. It demonstrates practical concepts such as process creation, process identification, process execution, signal handling, process synchronization, and process termination.

The project aims to demonstrate that a lightweight process and service supervisor can provide an organized and efficient mechanism for monitoring process states and automatically responding to process failures in a Linux environment.

3. Objectives

1. To understand the concepts of **Linux process and service management**.
2. To create and manage processes using system calls such as `fork()` and `exec()`.
3. To identify and monitor processes using mechanisms such as `getpid()`, `getppid()`, and process status information.
4. To implement process control operations such as **start, stop, restart, pause, resume, and terminate**.
5. To use Linux signals for communicating with and controlling managed processes.
6. To detect process termination or failure and implement an appropriate restart mechanism.
7. To study process synchronization using system calls such as `wait()` and `waitpid()`.
8. To develop a simple centralized interface for monitoring and controlling managed Linux processes.

4. Proposed Methodology

The project will be designed and implemented as a **Linux-based C/C++ application** that supervises selected processes and services.

1. **Process/Service Configuration:** The supervisor will maintain information about the processes or services that need to be monitored, including their executable commands and supervision parameters.
2. **Process Creation:** The supervisor will create managed processes using the `fork()` system call. The child process can then execute the required application using an appropriate `exec()` system call.
3. **Process Identification:** Each managed process will be identified using its **Process ID (PID)**. The supervisor will maintain the PID and relevant state information to monitor and control the process.

4. **Process Monitoring:** The supervisor will periodically check whether managed processes are running and maintain their current state. Process information can be obtained using Linux process-management mechanisms.
5. **Process Control:** The supervisor will provide operations such as **start, stop, restart, pause, resume, and terminate**. Linux signals such as SIGTERM, SIGKILL, SIGSTOP, and SIGCONT can be used for process control.
6. **Failure Detection and Restart:** If a supervised process terminates unexpectedly, the supervisor will detect the termination and can restart the process according to the configured supervision policy.
7. **Synchronization:** The supervisor will use mechanisms such as wait() or waitpid() to monitor child processes and correctly handle their termination and exit status.
8. **Logging and Status Reporting:** The system will maintain useful status information such as process PID, current state, start/stop events, termination events, and restart events. This information will help users understand the behavior of supervised processes

9. **Execution Flow:** The overall flow will be:

Load Configuration → Start Supervisor → Create/Start Process → Monitor Process → Detect State/Failure → Perform Required Action → Log Event → Restart/Stop → Continue Monitoring

10. **Testing:** The application will be executed in a Linux environment and tested using different process states and control operations to verify process monitoring, service control, failure detection, synchronization, and restart functionality.

5. Operating Systems Concepts / Linux APIs Used

OS Concept / Linux API / System Call	Purpose in the Project
Process Creation – fork()	Creates child processes that can be managed by the supervisor.
Program Execution – exec()	Loads and executes the selected application or service.
Process Identification – getpid(), getppid()	Retrieves process and parent process IDs for identification and management.
Signal Handling – signal() / sigaction()	Handles signals and allows the supervisor to respond to process events.
Signal Sending – kill()	Sends signals to managed processes for control operations.
Process Synchronization – wait()	Allows the supervisor to monitor child-process termination and exit status.
Process Control – SIGSTOP, SIGCONT, SIGTERM	Provides process termination, stopping, and resuming operations.
Process Monitoring – /proc	Provides Linux process information that can be used for monitoring process state.

6. Individual Contribution

Roll Number	Student Name	Individual Responsibility
2520030252	G. Jothi Ganesh	Process monitoring and status management
2520030179	V. Nitish Chowdhary	Process control, signals, and restart mechanism
2520030330	P.Santhosh	Interface, logging, testing, and documentation

7. Tools / Platforms / Software Used

Tool / Platform / Software	Purpose
Linux / Ubuntu	Provides the operating environment for developing, compiling, and executing the process communication program.
C / C++	Used to implement process creation, signal handling, communication, and process control.
GCC / G++ Compiler	Compiles the C/C++ source code into an executable program.
POSIX Libraries	Provides APIs and functions for signals, processes, and synchronization.
Linux /proc Filesystem	Provides process information used for monitoring managed processes.
Linux Terminal / Shell	Used to compile, execute, test, and monitor processes and signals.

8. Expected Outcome

The project is expected to produce a functional **Linux Service and Process Supervisor** that demonstrates how processes and services can be monitored and controlled in a Linux environment. The program will successfully create **parent and child processes** using `fork()`.

1. The supervisor will successfully **start and manage processes**.
2. Each managed process will be identified using its **PID and relevant process information**.
3. The supervisor will be able to **monitor the state of managed processes**.
4. The system will provide process-control operations such as **start, stop, restart, pause, resume, and terminate**.
5. Linux signals will be used to communicate with and control managed processes.
6. The supervisor will detect unexpected process termination and can **restart failed processes** according to the configured policy.
7. The supervisor will detect unexpected process termination and can **restart failed processes** according to the configured policy.
8. The project will provide practical understanding of **Linux process management, signals, synchronization, system calls, process monitoring, and system programming**.

9. GitHub Repository

GitHub Repository Link:

https://github.com/logicCrafter320/KLH_CSE_Y25_SEC9_22_Linux-Service-and-Process-Supervisor

Repository Created:

☒ Yes ☐ No

Faculty Name and Signature

Faculty Name:

Signature:

Remarks:

Faculty Use

Project Approved:

☐ Yes ☐ No

Date:

Faculty Signature: